Two-Pointer & Nested-Loop Patterns — Summary

Practice set: reverse string, palindrome check, two-sum (sorted), find duplicate

The Shared Skeleton (Two-Pointer Pattern)

Used in Problems 1–3. Only the middle line changes: swap, compare, or sum-check.

```
left, right = 0, len(x) - 1
while left < right:
    # swap / compare / check sum
    left += 1
    right -= 1
```

Mnemonic: two people walk toward each other from opposite ends of a line, doing one job as they pass.

Problem 1: Reverse a String

Pattern: Two Pointers (swap)

```
def get_rev(s):
    chars = list(s)          # need list: we are MODIFYING characters
    left, right = 0, len(chars) - 1
    while left < right:
        chars[left], chars[right] = chars[right], chars[left]
        left += 1
        right -= 1
    return ''.join(chars)

print(get_rev("hello"))   # olleh
```

Key rule: convert to list() only when you need to MODIFY characters. Reading/comparing only? No conversion needed.

Problem 2: Palindrome Check

Pattern: Two Pointers (compare, exit early)

```
def is_palindrome(s):
    left, right = 0, len(s) - 1
    while left < right:
        if s[left] != s[right]:
            return False
        left += 1
        right -= 1
    return True

print(is_palindrome("racecar"))   # True
print(is_palindrome("hello"))     # False
```

No list() needed here — only reading/comparing characters, never modifying them.

Problem 3: Two-Sum on a Sorted Array

Pattern: Two Pointers (sum check, converging)

```
def fun_sum_target(target, arr):
    l, r = 0, len(arr) - 1
    while l < r:
        t = arr[l] + arr[r]
```

```
        if t == target:
            return True
        elif t > target:
            r -= 1       # sum too big -> need smaller -> move right back
        else:
            l += 1       # sum too small -> need bigger -> move left forward
    return False

print(fun_sum_target(9, [2, 7, 11, 15]))   # True (2 + 7)
```

Works only because the array is SORTED — that's what lets you know which direction to move. Sum too small -> left moves forward (bigger numbers). Sum too big -> right moves backward (smaller numbers).

Problem 4: Find Duplicates in an Array

Pattern A: Nested Loop (brute force) — works but slow

```
def has_duplicate(arr):
    for i in range(len(arr)):
        for j in range(i + 1, len(arr)):
            if arr[i] == arr[j]:
                return True
    return False
```

Why not two pointers? No sorted order or mirror relationship -- a duplicate can be anywhere, so there's no direction to converge toward. That's the tell for nested loop instead.

Pattern B: Set (extra space) — faster, one pass

```
def has_duplicate(arr):
    seen = set()
    for num in arr:
        if num in seen:
            return True
        seen.add(num)
    return False

print(has_duplicate([1, 2, 3, 1]))   # True
```

Trigger words: "have I seen this before", "find duplicates", "count occurrences" -> reach for a set/dict, not nested loops.

Return vs Print — the common mix-up

return	print
Sends the value BACK to the caller, so other code can use it	Just DISPLAYS something on screen for a human to see
Needed so if/comparisons work: if has_duplicate(arr):	Returns None -- breaks if used inside an if check
Function's job: compute + return	Your job, separately: print(...) if you want to SEE the result

To see a result: result = is_palindrome("racecar"); print(result) -- or directly: print(is_palindrome("racecar"))

Problem 5: Find Maximum in an Array

Pattern: Single Pass (track one running value)

```
def comparator(arr):
    biggest = arr[0]        # start with first element, not 0
                            # (array could be all negative numbers)
    for i in range(1, len(arr)):
        if arr[i] > biggest:
            biggest = arr[i]
    return biggest

print(comparator([3, 7, 2, 9, 4]))   # 9
```

Key rule: start your tracker at arr[0], not 0 -- 0 can fail on all-negative arrays. Only remember ONE running value, not a full collection (that's what makes this single-pass, not extra-space).

Problem 6: Find Minimum in an Array

Pattern: Single Pass (same as Problem 5, flipped comparison)

```
def comparator(arr):
    smallest = arr[0]
    for i in range(1, len(arr)):
        if arr[i] < smallest:     # flipped: looking for SMALLER
            smallest = arr[i]
    return smallest

print(comparator([5, 1, 8, 3]))   # 1
```

Common bug: flipping the comparison direction (> vs <). Always trace through one example by hand before trusting the condition.

Problem 7: Find Min AND Max in One Pass

Pattern: Single Pass (track two running values together)

```
def comparator(arr):
    smallest = arr[0]
    biggest = arr[0]
    for i in range(1, len(arr)):
        if arr[i] < smallest:
            smallest = arr[i]
        if arr[i] > biggest:
            biggest = arr[i]
    return smallest, biggest

print(comparator([5, 1, 8, 3]))   # (1, 8)
```

No need for two separate loops -- both checks can happen in the same pass over the array, since they don't depend on each other.

Problem 8: Count Frequency of Each Number

Pattern: Extra Space (dict, not just a set)

```
def find_freq(arr):
    count = {}
    for num in arr:
        count[num] = count.get(num, 0) + 1
    return count

print(find_freq([1, 2, 2, 3, 1, 1]))
```

```
# {1: 3, 2: 2, 3: 1}
```

count.get(num, 0) means: get the current count, or 0 if this key doesn't exist yet -- avoids needing an if/else to check membership first. Common bug: writing count[num] = +1 (overwrites to 1) instead of count[num] = count[num] + 1 (adds to existing).

Avoid naming a dict variable 'dict' or a sum variable 'sum' -- these are built-in Python names and get shadowed. Use count, freq, total, etc. instead.

Checklist: Which Pattern to Reach For

1. Two Pointers -- mirror/reverse problems, or a SORTED array needing a pair/range
2. Single Pass -- track something (sum, max, count) walking through once
3. Nested Loop -- compare every element to every other (no sortedness/structure to exploit)
4. Extra Space (set/dict) -- "have I seen this", frequency counting, duplicates -- trade memory for speed